DATA MODELING · DOMAIN 10 - DATA MODELING

Dimensional models on the lakehouse

Section 10. The last teaching module. Facts, dimensions, keys, and which constraints are real.

SECTION 10 of 10 WEIGHT 6% TOPIC Dimensional models

admin@sthithapragna.com

The lakehouse did not abandon structure.
It made structure cheaper to change.

Where a dimensional model lives

- **Bronze** holds raw ingested data, close to the source and append-only.
- **Silver** holds cleansed and conformed data, with quality rules applied and entities resolved.
- **Gold** holds business-level data shaped for consumption, and **this is where dimensional models live**.
- So the medallion architecture and dimensional modelling are not alternatives. One describes the journey, the other the destination.

> ON THE EXAM
>
> A common confusion. Medallion is a layering strategy; star schema is a modelling technique. Your gold layer is usually a star schema.

Facts and dimensions

	CHARACTER
Fact table	Measurements and events. Long and narrow, growing constantly. Mostly keys and numbers.
Dimension table	Descriptive context. Wide and comparatively short, changing slowly.
Fact grain	What one row means. The single most important design decision.
Dimension design	Denormalised deliberately, to avoid joins at query time.

ON THE EXAM

Grain first. Deciding what one row of the fact table represents settles almost every other question about the model.

Why dimensions repeat themselves

- Databricks states it plainly: **normalisation maximises write integrity and storage efficiency**.
- **Denormalisation maximises read speed.** Star schemas duplicate descriptive attributes to optimise scan performance.
- On a lakehouse, storage is cheap and analyst time is not, so the trade usually favours reading.
- The cost is that an attribute changing means updating it in more than one place.

> ON THE EXAM
>
> A deliberate trade rather than sloppiness. A dimension repeating a region name a million times is doing that on purpose.

Surrogate and business keys do different jobs

- A **business key** comes from the source system. A **surrogate key** is system-generated and meaningless.
- Surrogates protect you when **business keys collide across source systems**, which they eventually do.
- In a Type 2 dimension: **the business key tells you who, and the surrogate key tells you which version of them**.
- Facts join on the surrogate, so a fact row is tied to the version of the dimension that was current at the time.

> ON THE EXAM
>
> That third line is the clearest statement of why Type 2 dimensions need surrogate keys at all.

Identity columns, and the two forms

- An identity column generates a sequential value automatically. It is the standard surrogate key mechanism.
- **Generated always** prohibits inserting your own values. **Generated by default** permits them.
- Always is safer for a true surrogate. By default is useful when migrating existing keys in.
- **Gaps in the sequence are normal and do not matter**, because the value is meaningless by design.

> ON THE EXAM
>
> People worry about gaps. A surrogate key has no meaning, so a gap has no meaning either.

Identity columns and AUTO CDC do not combine

- **Identity columns are not supported on AUTO CDC targets**, from module 8.
- They may also be **recomputed during materialized view updates**, so Databricks recommends them only with streaming tables.
- So the standard surrogate key mechanism collides with the standard SCD mechanism.
- The documented alternative: a **deterministic hash of the business key** and the attributes that define a version.

> ON THE EXAM
>
> Worth knowing because both halves are recommended practice, and they do not work together. A hash sidesteps it entirely.

Primary and foreign keys are informational

- Supported since **DBR 11.3 LTS**, generally available as of **15.2**.
- **They are informational and not enforced at write time.** A violating row is accepted.
- They still earn their place: they **document relationships**, the **optimizer can use them**, and BI tools discover joins from them.
- Databricks says **enforceable referential integrity is on the roadmap**.

> ON THE EXAM
>
> Not enforced does not mean not useful. It means the guarantee has to come from your pipeline rather than from the table.

Two kinds of constraint on the same table

- **Primary key and foreign key: informational.** Not enforced. Documentation and optimisation.
- **NOT NULL and CHECK: enforced.** A violating write fails, from module 32.
- So a Delta table carries both kinds at once, and knowing which is which is directly examinable.
- If referential integrity actually matters, it must be guaranteed by your pipeline, not by the constraint.

> ON THE EXAM
>
> The single most likely confusion in this module. Enforcement is per constraint type, not a table-wide property.

The dimension types, in one place

- **Type 1** overwrites. The dimension holds current truth and no history.
- **Type 2** adds a row per version, with start and end columns, and a null end meaning current.
- Module 8's AUTO CDC implements both, sequenced by a column you nominate rather than by arrival order.
- Choosing is a business question: does anybody need to know what was true on a past date?

> ON THE EXAM
>
> The technique is settled. The decision is not, and it belongs to whoever owns the reporting requirement.

Missing members and degenerate dimensions

- A **missing member** is a dimension row representing unknown or not applicable, so facts always have something to join to.
- That keeps joins inner rather than outer, and stops unmatched facts vanishing from a report.
- A **degenerate dimension** is an attribute kept in the fact table rather than given its own dimension.
- Typically an identifier like an order number, where a separate table would hold nothing but the key.

> ON THE EXAM
>
> Both are named patterns in Databricks' own material, which makes their vocabulary fair game in a question.

How the model meets the storage

- Fact tables are large, frequently filtered and joined, which is exactly what **liquid clustering** suits.
- Cluster on the columns queries filter and join on, most often a date and one or two dimension keys.
- Remember the limit: **up to four clustering keys**, and clustering on unrelated columns dilutes the benefit.
- Dimension tables are usually small enough that layout barely matters.

> ON THE EXAM
>
> Module 22's material applied to a model. The fact table is where layout decisions pay, and the dimensions rarely need them.

Why this is not a legacy topic

- **Metric views depend entirely on well-structured relational models** to work, and they are a current feature.
- Semantic layers need proper facts and dimensions to deliver consistent metrics across an organisation.
- Databricks also notes that AI and machine learning perform best on clean, structured, tabular data.
- Use the **three-level namespace** to organise the model, which is module 27's catalog, schema and table.

> ON THE EXAM
>
> The argument for dimensional modelling on Databricks is a forward-looking one, not nostalgia for the warehouse.

What to carry into the exam

- **Gold is where dimensional models live.** Medallion is layering; star schema is modelling. Not alternatives.
- **PK and FK are informational and not enforced. NOT NULL and CHECK are enforced.**
- Identity columns generate surrogates. **Generated always prohibits your own values; by default permits them.**
- **Identity columns are not supported on AUTO CDC targets.** Hash the business key instead.
- Business key says who; surrogate key says **which version**. Facts join on the surrogate.

> ON THE EXAM
>
> That completes the teaching content: ten sections, thirty-two modules. Two cross-cutting modules and three practice exams remain.

> A team defines a foreign key constraint from a fact table to a dimension table. They then remove the referential integrity checks from their pipeline, reasoning that the constraint now enforces it.
>
> Three months later, reports show a growing number of transactions attributed to an unknown customer.

What went wrong?

A The constraint was defined with the wrong syntax and was never applied.

B Primary and foreign key constraints are informational and not enforced at write time. The pipeline still has to guarantee integrity.

C Foreign keys are only enforced on managed tables.

D The constraint requires the dimension to have a NOT NULL primary key to take effect.

◆ PAUSE AND DECIDE

What went wrong?

A The constraint was defined with the wrong syntax and was never applied.

An invalid constraint would have errored at definition. This one is working as designed.

B Primary and foreign key constraints are informational and not enforced at write time. The pipeline still has to guarantee integrity.

Not enforced means violating rows are accepted silently, which is exactly the symptom.

C Foreign keys are only enforced on managed tables.

They are not enforced on any table type.

D The constraint requires the dimension to have a NOT NULL primary key to take effect.

No configuration makes a foreign key enforced.

ON THE EXAM

The constraint did its job, which was documentation and optimisation. It was never going to reject a row.

> A team builds a Type 2 customer dimension using AUTO CDC, and defines the surrogate key as an identity column. The pipeline fails to deploy.
>
> They need a stable surrogate key that identifies each version of a customer.

What is the problem and the alternative?

A. Identity columns are not supported on AUTO CDC targets. Generate the surrogate as a deterministic hash of the business key and versioning attributes.

B. Type 2 dimensions cannot have surrogate keys; use the business key instead.

C. The identity column must be declared as generated by default rather than generated always.

D. AUTO CDC does not support Type 2; use MERGE.

◆ PAUSE AND DECIDE

What is the problem and the alternative?

A Identity columns are not supported on AUTO CDC targets. Generate the surrogate as a deterministic hash of the business key and versioning attributes.

Two recommended practices that do not combine, with a documented alternative.

B Type 2 dimensions cannot have surrogate keys; use the business key instead.

The business key cannot identify a version. That is what the surrogate is for.

C The identity column must be declared as generated by default rather than generated always.

Neither form is supported on an AUTO CDC target.

D AUTO CDC does not support Type 2; use MERGE.

AUTO CDC supports Type 2 directly, and MERGE cannot sequence out-of-order changes.

ON THE EXAM

Two things Databricks recommends separately that do not work together. Recognising the collision is the skill.

> A report of sales by region is missing about 3% of total revenue compared to the raw fact table. Investigation shows those fact rows have a region key that does not appear in the region dimension, because the source system occasionally sends a blank region.
>
> The report uses an inner join to the dimension.

What is the standard modelling fix?

A. Change the report to a left outer join and handle the nulls in the reporting tool.

B. Add a missing member row to the dimension, and map unmatched facts to it during load.

C. Add a foreign key constraint so the bad rows are rejected at write time.

D. Filter the blank regions out of the fact table during ingestion.

◆ PAUSE AND DECIDE

What is the standard modelling fix?

A. Change the report to a left outer join and handle the nulls in the reporting tool.
Pushes the problem into every report that ever joins this dimension.

B. Add a missing member row to the dimension, and map unmatched facts to it during load.
The named pattern. Every fact has something to join to, revenue reconciles, and the gap is visible as its own category.

C. Add a foreign key constraint so the bad rows are rejected at write time.
Foreign keys are informational and reject nothing.

D. Filter the blank regions out of the fact table during ingestion.
Makes the numbers reconcile by deleting revenue, which is worse than the symptom.

ON THE EXAM

The missing member turns silently vanishing rows into a visible category called Unknown, which is what you want a report to show.

> A team is migrating a customer dimension from an existing warehouse. They need to preserve the surrogate keys already used by historic fact rows, then generate new keys for customers added afterwards.
>
> They plan to use an identity column.

Which form should they use?

A Generated always, which guarantees the keys are unique.

B Generated by default, which permits inserting the existing keys during migration and generates new ones afterwards.

C Neither; identity columns cannot coexist with pre-existing key values.

D Generated always, then update the keys afterwards to match the originals.

◆ PAUSE AND DECIDE

Which form should they use?

A Generated always, which guarantees the keys are unique.

It prohibits inserting your own values, which is exactly what the migration requires.

B Generated by default, which permits inserting the existing keys during migration and generates new ones afterwards.

The difference between the two forms is precisely whether your own values may be supplied.

C Neither; identity columns cannot coexist with pre-existing key values.

That is what the by-default form exists for.

D Generated always, then update the keys afterwards to match the originals.

Rewrites every key after loading, and breaks the fact joins in between.

ON THE EXAM

The two forms exist for exactly this distinction. Always for a clean new dimension; by default when you are bringing keys with you.

> A gold fact table holding four billion rows is queried mostly by date range and by store. Query times are poor. The table is unpartitioned and unclustered, and is a Unity Catalog managed table.
>
> A team member proposes clustering on eight columns to cover every query pattern the analysts use.

What is the better approach?

A Cluster on all eight columns, since more coverage means more queries benefit.

B Cluster on the columns actually filtered and joined on, within the four-key limit, or use automatic clustering.

C Partition by date and Z-order by store, which covers both access patterns.

D Leave it unclustered; predictive optimization will handle layout automatically.

◆ PAUSE AND DECIDE

What is the better approach?

A Cluster on all eight columns, since more coverage means more queries benefit.

Exceeds the four-key limit, and clustering on unrelated columns dilutes the benefit for all of them.

B Cluster on the columns actually filtered and joined on, within the four-key limit, or use automatic clustering.

Date and store are named in the scenario, and automatic clustering is available on a managed table with predictive optimization.

C Partition by date and Z-order by store, which covers both access patterns.

Clustering replaces both, and Z-ordered files are ignored by predictive optimization.

D Leave it unclustered; predictive optimization will handle layout automatically.

It maintains tables and compacts files. Choosing clustering keys needs CLUSTER BY AUTO.

ON THE EXAM

The scenario names the access pattern, which is the answer. Everything else is the team member guessing at coverage.

> A team argues their gold layer should be a single wide denormalised table rather than a star schema. Their reasoning is that joins are expensive and storage is cheap.
>
> The business needs consistent metric definitions across several reporting tools, and wants to use metric views.

What is the strongest counter-argument?

A. A single wide table is always slower to query than a star schema.

B. Metric views and semantic layers depend on well-structured relational models with proper facts and dimensions.

C. Denormalisation is not supported on Delta tables.

D. Wide tables cannot exceed the column limit for Unity Catalog tables.

◆ PAUSE AND DECIDE

What is the strongest counter-argument?

A. A single wide table is always slower to query than a star schema.
Overstated. A wide table can be very fast for the queries it was shaped for.

B. Metric views and semantic layers depend on well-structured relational models with proper facts and dimensions.
Databricks states this directly, and the scenario names metric views as a requirement.

C. Denormalisation is not supported on Delta tables.
Star schemas denormalise dimensions deliberately. It is a technique, not a restriction.

D. Wide tables cannot exceed the column limit for Unity Catalog tables.
A limit exists in principle and is not the argument here.

ON THE EXAM

The team's reasoning about joins and storage is sound. It just does not address the requirement the business actually stated.